Free Plugin Adds Voice to AI Coding Agents — Developers Hear When Tasks Finish or Need Approval

punt-vox delivers spoken notifications, on-demand summaries, and background music through five TTS providers — including zero-config offline fallbacks — so developers can multitask while their agent works, even across machines

Press Release

SAN FRANCISCO — September 2026 — Today, punt-labs announced the general availability of punt-vox, a free, open-source plugin that gives AI coding agents a voice. When Claude Code finishes a task, hits an error, or gets stuck waiting for permission, the developer hears it — through premium text-to-speech from ElevenLabs, OpenAI, or AWS Polly, with zero-config offline fallbacks via macOS `say` and Linux `espeak-ng`. punt-vox delivers natural-sounding voice through a properly packaged Python library with a one-command install, zero configuration, and graceful absence when not installed. For developers with multi-machine setups, punt-vox supports remote audio: run Claude Code on a headless server and hear notifications on the local workstation (see FAQ 2).

Problem

Developers who use AI coding agents multitask. They start a refactor, switch to a document, answer a Slack thread. The agent keeps working — for seconds, sometimes for thirty minutes or more on autonomous tasks [1]. When it finishes, or when it needs permission to proceed, the developer does not know until they check the terminal. On complex tasks, the agent stops to ask for clarification more than twice as often as the developer interrupts it [2]. While auto-approve rates are growing — from 20% for new users to over 40% for experienced users — the majority of sessions still involve at least one approval gate, and each unnoticed prompt is dead time: the agent waits, the developer works on something else, and neither makes progress on the original task.

The cost is cumulative. Reactive interruptions cost over 23 minutes of recovery time on average [3]. Self-initiated checks — glancing at the terminal to see if the agent is done — are lower-cost than reactive interruptions, but each check still breaks the developer's current focus. Across a day of agentic work, these small breaks add up: 51% of professional developers now use AI tools daily [4], and a growing fraction use autonomous agents that run for extended periods (see FAQ 8).

Solution

punt-vox adds an audio layer to Claude Code through slash commands.

`/vox y` enables chime notifications for two events: task completion and permission prompts. When the agent finishes a multi-file refactor, the developer hears an audio tone. When the agent needs approval to run a destructive command, the developer hears that too — no more silent permission dialogs burning minutes of idle time. `/vox c` adds continuous mode with spoken summaries: milestone updates during long-running tasks, so the developer hears progress without checking in. `/unmute` enables voice mode with a specific voice from the roster.

`/mute` switches from voice to chime — the same events fire, but the notification is an audio tone instead of words. Chimes are mood-aware: when a vibe is active, tones pitch-shift to match the session mood (brighter when things are going well, darker when they're not). Eight distinct signals across three mood variants keep chime-only mode as expressive as voice. For developers who want awareness without narration.

`/recap` is on-demand: after a wall of terminal output, the developer types `/recap` and hears a 30-second spoken summary of what just happened. No scrolling, no parsing diffs. The agent extracts the key points and speaks them while the developer scans the code (see FAQ 3).

`/music on` starts vibe-driven background music that loops during coding sessions. The music adapts when the session mood changes: upbeat during green test runs, subdued during debugging. Music plays at reduced volume under speech and chimes.

The plugin auto-detects the best available TTS provider — ElevenLabs for natural voice quality, OpenAI for low latency, AWS Polly for cost efficiency, macOS `say` and Linux `espeak-ng` for zero-config offline use — and requires no voice selection or audio device configuration. For developers who SSH into remote machines, setting `VOXD_HOST` and `VOXD_PORT` routes audio to a local `voxd` daemon so notifications play on the workstation, not the server (see FAQ 2).

Customer Quote

> *"I run Claude Code on a second monitor for refactoring work. Before punt-vox, I'd context-switch back every few minutes to check if it was done or stuck on a permission prompt. The worst was realizing it had been waiting for approval for twenty minutes while I was in a design review. Now I hear 'task complete' or 'needs your approval' and I know exactly when to switch back. Last Thursday I went through a full hour-long code review without once checking the terminal — and I didn't miss a single prompt."*
>
> — **Marcus Chen**, Senior Software Engineer at a Series B startup

Getting Started

One command installs everything:

```
curl -fsSL https://raw.githubusercontent.com/punt-labs/vox/main/install.sh | sh
```

The script checks Python 3.13+, installs `uv` if missing, installs the CLI, registers the MCP server with Claude Code, and runs `vox doctor` to verify that at least one TTS provider is available. Restart Claude Code and type `/vox y`. From that moment, every task completion and permission prompt produces a chime notification. Type `/vox c` for continuous spoken summaries.

There is no voice selection wizard and no audio device configuration. The `voxd` daemon handles synthesis, caching, and playback as a system service. If `vox doctor` passes, everything works. If punt-vox is not installed, Claude Code behaves exactly as before — no error, no degradation.

Spokesperson Quote

> "We built punt-vox because we kept walking away from the terminal and losing time. The agent finishes, or worse, it sits waiting for permission, and nobody knows. Voice solves this in a way that visual notifications cannot — it reaches you when your eyes are on another screen. We chose premium TTS providers because the OS built-in voice is unpleasant enough that people turn it off, which defeats the purpose. If the notification sounds good, people leave it on. That is the entire design philosophy: voice that supplements text, never replaces it, and sounds good enough to keep."
>
> — **JM Freeman**, Creator of punt-vox

Call to Action

punt-vox is available now at `https://github.com/punt-labs/vox`. It is free, open source (MIT license), and always will be. Install with one command:

```
curl -fsSL https://raw.githubusercontent.com/punt-labs/vox/main/install.sh | sh
```

Restart Claude Code and type `/vox y`.

Frequently Asked Questions

External FAQs

Q1: What is punt-vox and who is it for?

punt-vox is a voice notification layer for AI coding agents. It is for developers who use Claude Code for autonomous tasks — refactoring, debugging, code generation — and want to know when the agent finishes or needs input without watching the terminal. If you multitask while your agent works, punt-vox is for you. This includes developers who run Claude Code on remote servers via SSH: punt-vox's remote audio support routes notifications to the local workstation where the developer can hear them.

Q2: How is this different from agent-notify or AgentVibes?

Two open-source tools already add notifications to coding agents. agent-notify [5] provides sound alerts and macOS `say` voice using the operating system's built-in speech synthesizer. AgentVibes [6] is more capable: 914 voices via macOS `say`, Piper TTS, and Soprano, plus background music and personality styles. A third project (claude-code-voice-handler) used OpenAI TTS for Claude Code notifications but appears to have been taken down, suggesting maintenance risk is real in this space.

punt-vox differs in three ways:

Voice quality. Most existing tools use free, local TTS engines. punt-vox uses ElevenLabs (consistently among the top-rated providers on the HuggingFace TTS Arena [7]), OpenAI TTS, and AWS Polly — voices that sound natural enough to leave on.

Packaging. punt-vox is a PyPI package with a test suite, strict type checking, an MCP server, and a documented API. It is installed with `uv tool install` and configured with `vox install`. Existing tools are GitHub repositories without distribution packaging.

Separation of concerns. punt-vox is a TTS engine that other tools can build on. langlearn-tts (language learning audio) already uses punt-vox as its speech backend. The MCP server exposes synthesis, batching, and pair stitching as tools that any MCP client can call.

Since v0.7.0, punt-vox also includes zero-config offline fallbacks — macOS `say` and Linux `espeak-ng` — so it works with no API key at all, matching the accessibility of existing tools. The premium providers (ElevenLabs, OpenAI, Polly) require an API key but deliver voice quality that the offline fallbacks cannot match.

Q3: How do I get started?

One command:

```
curl -fsSL https://raw.githubusercontent.com/punt-labs/vox/main/install.sh | sh
```

The script installs the CLI, registers the MCP server with Claude Code, and runs `vox doctor` to verify providers. Restart Claude Code. Type `/vox y`. Notifications are now active. Type `/recap` after any long output for a spoken summary.

For manual installation: `uv tool install punt-vox && vox install && vox doctor`.

Q4: How much does it cost?

punt-vox itself is free (MIT license). TTS provider API calls have costs:

- ElevenLabs: from $5/month for 30,000 characters [8]
- OpenAI TTS: $15 per million characters (tts-1) [8]

- AWS Polly: $4.80 per million characters (Standard) [8]

A typical notification is 50–200 characters. At OpenAI's rate, 1,000 notifications cost approximately $0.003. Even heavy daily use (50 notifications/day) runs under $0.10/month. The dominant cost is the API key setup, not the usage.

Q5: What happens to my data?

punt-vox sends the text of each notification to your chosen TTS provider for synthesis. The provider receives the text, returns audio, and punt-vox plays it locally. No data is stored by punt-vox beyond a local audio cache in `~/.punt-labs/vox/cache/` and saved audio in `~/Music/vox/` (configurable via `VOX_OUTPUT_DIR`). punt-vox has no telemetry, no analytics, and no cloud service of its own.

The privacy implication: your TTS provider sees the text of your notifications. For ElevenLabs and OpenAI, review their data retention policies. For AWS Polly, standard AWS data handling applies. If this is unacceptable, punt-vox is not the right tool — the existing OS TTS alternatives process text locally with no external API calls.

Q6: Does this work with agents other than Claude Code?

punt-vox is a standalone TTS engine distributed as a Python library and MCP server. The Claude Code plugin (`/vox`, `/unmute`, `/mute`, `/recap`) is specific to Claude Code's hooks system. The underlying TTS engine can be called from any Python program or any MCP-compatible client. Integration with Cursor, GitHub Copilot, or other agents would require writing hooks for those tools' notification systems.

Q7: Can I use my own voice or choose a specific voice?

punt-vox auto-selects a default voice per provider (Matilda for ElevenLabs, Nova for OpenAI, Joanna for Polly). Use `/unmute @matilda` to set a session voice, or `/unmute @` to browse the roster. The CLI accepts `--voice` flags for overriding the default. The design philosophy is zero configuration — defaults work out of the box, but voice selection is available for those who want it.

Internal FAQs

Value & Market

Q8: What is the total addressable market?

Bottoms-up estimate from three data points:
AI coding agent users: 31% of professional developers use AI agents [4]. GitHub Copilot alone has 20 million cumulative users [9]. Cursor has over 1 million users [10]. Claude Code is used by 41% of AI agent users [4].
Autonomous task runners: Not all agent users run autonomous tasks. Anthropic data shows the median Claude Code turn is approximately 45 seconds, but the 99.9th percentile grew from under 25 minutes to over 45 minutes between October 2025 and January 2026 [2]. The growth is driven by user behavior — granting more autonomy — not just model capability. Background tasks are explicitly recommended for work exceeding 30 seconds [11].
Target segment: Developers who run autonomous agent tasks and multitask during execution. Applying the math: if roughly 28 million professional developers exist globally, 31% use agents (8.7M), 41% of agent users use Claude Code (3.6M), and 10–20% run autonomous tasks long enough to warrant notification, the target segment is 360,000 to 720,000 developers. This is the upper bound; actual reachable users via PyPI are a fraction of this.
Because punt-vox is free, TAM is measured in adoption, not revenue. Success means becoming the default voice layer that Claude Code users install alongside the agent.

Q9: What evidence do we have that developers want this?

Three categories of evidence, all indirect. No primary customer data exists yet (hypothesis stage).
Behavioral: Two open-source tools already exist to solve this problem. agent-notify [5] adds sound and voice notifications to Claude Code, Cursor, and Codex. AgentVibes [6] adds voice, background music, and personality styles to Claude Code. The existence of these tools — built by independent developers, not companies — is evidence that the pain point is real enough for people to build solutions.
Structural: Anthropic's own data confirms the structural conditions. Agents ask for clarification more than twice as often as users interrupt on complex tasks [2]. The hooks system exposes `Notification` events with `permission_prompt` and `idle_prompt` subtypes [12] — Anthropic built the infrastructure for exactly this kind of notification.
Adjacent: Research on audio notifications shows they maintain workspace awareness without demanding visual attention [13]. IDC projects 80% of enterprise apps will embed AI copilots by end of 2026 [14], increasing the notification volume that will compete for developer attention. Voice occupies a different attentional channel than visual notifications.
What is missing: No user interviews. No usage data. No measurement of how long permission prompts sit unnoticed. This is the most important research gap (see FAQ 12).

Q10: Who are the competitors and why will we win?

agent-notify [5]: Open-source. Sound alerts, macOS `say`, ntfy push notifications. Supports Claude Code, Cursor, Codex. Strengths: no API key required, cross-agent support. Weakness: OS TTS only, no packaging or test suite, single developer.
AgentVibes [6]: Open-source Claude Code plugin. macOS `say`, Piper TTS, Soprano neural TTS, Windows SAPI. 914 voices. Background music, personality styles, verbosity levels.

Strengths: most feature-complete, creative UX (personalities, music). Weakness: no premium TTS providers (no ElevenLabs, no OpenAI TTS), no PyPI packaging, no test suite.

OS notification center: macOS/Linux system notifications. Always available, no setup. Weakness: visual-only, competes with every other app's notifications, easy to miss during focus work.

Why punt-vox wins: Voice quality. The macOS `say` voice is recognizable and unpleasant enough that users turn it off. ElevenLabs consistently ranks among the top TTS providers in blind listening tests and has $330M+ ARR [7]. If the notification sounds good, people leave it on. This is the core bet: premium voice quality drives sustained adoption where OS TTS does not.

Honest assessment of risk: If AgentVibes adds ElevenLabs support, it becomes a strong competitor with a creative feature set (personalities, verbosity levels). punt-vox now also has background music (v4.4.0 via ElevenLabs Music API) and zero-config offline TTS (macOS `say` and Linux `espeak-ng`), narrowing the feature gap. punt-vox's defense is engineering quality (packaged, tested, typed, reusable as a library), separation of concerns (TTS engine vs. plugin UX), and distributed audio (remote `voxd`). The competitive moat is narrow.

Q11: What is the user acquisition strategy?

Three channels:

PyPI discoverability: Users searching for "claude code voice" or "tts mcp" find punt-vox. The install script auto-configures MCP, reducing setup to one command.

Claude Code plugin marketplace: When the marketplace launches, punt-vox will be listed as a plugin. One-click install with `/install punt-vox`.

Word of mouth: Voice notifications are inherently demonstrable. When a developer's agent speaks "task complete" during a pairing session or meeting, the person next to them asks what that was. This is the strongest acquisition vector for an audio product.

No paid acquisition planned. The product is free; the audience is technical; organic distribution is the appropriate channel.

Q12: What is your next step to validate your vision?

All core features are shipped (v4.7.6). The product is approaching public recommendation. Validation now focuses on adoption and retention:

1. Do users who enable `/vox y` leave it on after one week?
2. Which event fires more often: task completion or permission prompt?
3. Do users stay on chimes (`/vox y`) or upgrade to voice (`/unmute`)?
4. Does remote `voxd` expand the addressable audience to SSH-based workflows?

If retention after one week is below 30%, the value proposition is wrong and we should stop. If permission-prompt notifications dominate, that reframes the product from "voice for your agent" to "never miss a permission prompt again." Both outcomes inform whether to continue.

Target: 5 developers using punt-vox daily for 2 weeks. Collect feedback via GitHub Discussions.

Technical

Q13: What are the major technical risks?

Claude Code hooks stability (medium risk): punt-vox depends on Claude Code's hooks system — specifically the `Stop` event and `Notification` event with `permission_prompt` subtype [12]. Hooks are documented first-party API, but breaking changes could disable notifications. Mitigation: hooks are a foundational feature of Claude Code's extensibility model, used by many plugins. Breaking them would affect the entire plugin ecosystem.

TTS provider latency (low risk): Voice notifications must arrive within 1–2 seconds of the triggering event to feel real-time. ElevenLabs streaming API and OpenAI TTS both return first audio within 200–500ms. AWS Polly is faster still. Mitigation: auto-detect the lowest-latency available provider for notifications; reserve higher-quality providers for `/recap` summaries where latency is less critical.

Audio playback on headless/remote systems (resolved): Developers using Claude Code via SSH or in cloud VMs previously had no audio path. As of v4.7.7, `VOXD_HOST`/`VOXD_PORT`/`VOXD_TOKEN` env vars route synthesis requests to a remote `voxd` instance running on the developer's local workstation. Token-based authentication secures the connection. `vox doctor` reports active env var overrides. For local-only setups, `vox doctor` still checks audio playback capability at install time and notifications fail silently when audio is unavailable.

All core technical risks have been resolved. The TTS engine (synthesis, batching, merging) is built and tested. The Stop hook uses a decision-block pattern where the model generates a 1-2 sentence summary and calls the TTS tool. Vibe tags are resolved deterministically from accumulated session signals, written to a gitignored ephemeral config file (`vox.local.md`), and picked up automatically by the TTS tool — no data passes through user-visible output.

Q14: What dependencies exist on external systems?

Claude Code hooks: Required for `/vox` notifications to fire automatically. Without hooks, punt-vox still works as a manual TTS tool (`/recap`, CLI synthesis) but loses its primary value proposition.

TTS provider APIs: At least one of ElevenLabs, OpenAI, or AWS Polly must be reachable. Auto-detection falls back through the chain. If all providers are unavailable, `vox doctor` reports the failure and no audio plays.

pydub + ffmpeg: Used for audio merging and format conversion. ffmpeg is required at runtime. `vox doctor` checks for its presence.

MCP protocol: The plugin registers as an MCP server. MCP is maintained by Anthropic and adopted by multiple clients. Low risk of abandonment.

No runtime cloud dependencies exist beyond the TTS API calls themselves. punt-vox has no backend service, no database, and no telemetry. The optional remote `voxd` mode connects to a user-controlled daemon on another machine — not to any punt-labs service.

Q15: What is the estimated development timeline?

punt-vox is at v4.7.6 on PyPI (May 2026). The product is nearly feature-complete.

Shipped: Stop hook notification with spoken summaries. `/vox y|n|c` toggle. `/unmute`/`/mute` for voice vs. chime. Permission-blocked notification via `Notification` hook. `/recap` command (agent-executed spoken summary). `/vibe` with auto-detection and manual modes — auto-vibe uses deterministic signal-to-tag mapping (no LLM needed) to resolve ElevenLabs expressive tags from accumulated session signals. Five providers (ElevenLabs, OpenAI, Polly, macOS `say`, Linux `espeak-ng`). Mood-aware chimes with pitch shifting (8 signals × 3 moods). Non-blocking MCP tools. `voxd` audio daemon with WebSocket protocol (v3.0.0) — separates synthesis/playback

from MCP server for sub-10ms hook dispatch. Background music generation via ElevenLabs Music API (v4.4.0). Per-call API key passthrough for billing isolation (v4.3.0). Config split into tracked `vox.md` and ephemeral `vox.local.md` (v4.7.5). Claude Code plugin on marketplace. CLI global flags (`--json`, `--verbose`, `--quiet`).
In progress: Remote `voxd` connectivity — `VOXD_HOST`/`VOXD_PORT`/`VOXD_TOKEN` env vars for cross-host audio playback without SSH tunnels. Server-side `VOXD_BIND` controls the bind address (v4.7.7).
Next: Downstream library adoption beyond langlearn-tts. Public recommendation and outreach.

Q16: What is the scaling story?

punt-vox runs on the developer's machine. There is no server to scale. Each notification is an independent TTS API call — no state accumulates, no queues build up.
At the TTS provider level: ElevenLabs handles concurrent requests across millions of users ($330M+ ARR, 41% Fortune 500 adoption [7]). OpenAI and AWS Polly are similarly robust. punt-vox will never be a meaningful fraction of any provider's load.
The only local scaling concern is audio file accumulation. Saved audio lands in `~/Music/vox/` (configurable); the synthesis cache in `~/.punt-labs/vox/cache/` is content-addressed and can be cleared via `vox cache clear`.

Business

Q17: What is the revenue model?

There is none. punt-vox is free, open-source software under the MIT license. The project exists to solve a real problem in the Claude Code ecosystem and to serve as the TTS engine for other punt-labs tools (langlearn-tts uses punt-vox as its speech backend). Value is measured in adoption and ecosystem contribution, not revenue.

Q18: What does maintaining punt-vox cost?

Infrastructure: Zero. punt-vox runs on the user's machine. GitHub Actions CI is free for public repos.
TTS API costs during development: The test suite mocks all providers, so CI incurs zero TTS costs. Manual integration testing against live providers costs under $0.10 per session. Negligible.
Maintainer time: The primary cost. Estimated at 2–5 hours per week during active development (Phases 1–3), dropping to 1–2 hours per week for maintenance (bug fixes, dependency updates, community triage).
Opportunity cost: Maintainer time spent on punt-vox is time not spent on other punt-labs projects. If adoption metrics do not meet thresholds after 3 months (see FAQ 19), maintenance mode is the appropriate response — fix bugs, accept PRs, but no new features.

Q19: What are the key metrics for success?

- **PyPI installs:** Monthly download count as adoption proxy. Target: 500 monthly installs within 6 months of the notification feature shipping.
- **Notification retention:** Percentage of users who enable `/vox y` and keep it enabled after one week. Target: 30%+. Below this, the product is not sticky.

- **Event distribution:** Ratio of task-completion to permission-prompt notifications. Informs whether the product's primary value is "hear when done" or "never miss a prompt."
- **GitHub stars:** Community interest signal. Target: 200 stars within 6 months.
- **Downstream dependents:** Other tools using punt-vox as a library. Current: 1 (langlearn-tts). Target: 3+ within a year.

Decision threshold: if notification retention is below 30% after the initial user cohort, reconsider the product direction before investing in Phase 3 features.

Q20: Why now? What has changed?

Three converging shifts:

Autonomous agents are real. Claude Code's 99.9th percentile task duration grew from under 25 minutes to over 45 minutes in three months [2]. Background tasks are a documented workflow [11]. Agents now run long enough that developers routinely leave them unattended.

Hooks enable it. Claude Code's hooks system exposes lifecycle events — `Stop`, `Notification`, `TaskCompleted` — as structured, documented API [12]. Before hooks, notification plugins required polling or terminal scraping. Hooks make event-driven notifications clean and reliable.

Premium TTS is accessible. ElevenLabs, OpenAI, and Polly all offer API-based synthesis at costs below $0.01 per notification. Two years ago, natural-sounding TTS required expensive enterprise contracts. Today it costs less than a fraction of a cent per utterance [8].

Q21: What are we not building?

Covered in the Feature Appendix (Won't Do section). The boundaries that matter most:

- **Not voice input.** punt-vox speaks to the developer. It does not listen. Voice commands for agents are a different product.
- **Not a screen reader.** Accessibility narration of terminal output requires fundamentally different architecture (continuous reading, cursor tracking, ARIA-like semantics). punt-vox is event-driven notifications, not continuous narration.
- **Not personality entertainment.** No agent character voices, no conversational persona styles. punt-vox adapts to mood via vibe tags for *information clarity* and plays background music for *ambient focus*, but does not role-play characters.

Q22: What does failure look like?

Two plausible failure scenarios:

People turn it off. Voice notifications are intrusive. If the default voice is too loud, too frequent, or too slow, users disable `/vox n` within hours and never re-enable. The notification retention metric (see FAQ 19) is the canary: below 30% after one week means the UX is wrong. *Response:* `/mute` (chime-only) is the escape valve. If retention is low on voice but acceptable on chime, the product is an audio notification tool, not a voice tool. Adjust positioning accordingly.

The pain point shrinks. Anthropic's data shows auto-approve rates growing from 20% to 40%+ as users gain experience [2]. If the trend continues, permission prompts become rare and the strongest use case ("never miss a prompt") weakens. At the same time, longer autonomous runs make the task-completion notification more valuable. *Response:* Monitor the event distribution metric. If permission prompts decline to less than 10% of notifications, pivot messaging to "hear when your agent finishes" and invest in `/recap` quality.

Risk Assessment

Risk	Rating	Assessment
Value	**Medium**	Demand is proven by the existence of two competing tools. The bet is that premium voice quality drives sustained adoption where OS TTS does not. No primary user data yet.
Usability	**Low**	One-command install and auto-detection of providers. Since v0.7.0, offline fallbacks (macOS `say`, Linux `espeak-ng`) work with zero API keys — install and go. Premium providers still require account creation and environment variable configuration, but the onboarding path no longer requires it.
Feasibility	**Low**	The TTS engine, hooks integration, `voxd` daemon, background music, config split, and all commands are built and shipped. Remote audio (v4.7.7) is the last major capability in progress. Vibe tags are resolved deterministically from session signals — no open design questions remain.
Viability	**Low**	No revenue model to validate. Zero infrastructure cost. MIT license. The only cost is maintainer time.

Feature Appendix

Defines the scope boundary for punt-vox. Every feature is explicitly categorized to prevent scope creep and force prioritization decisions upfront.

Shipped

All features essential for the core value proposition are delivered and available on PyPI.

F1. Stop hook notification — When Claude Code finishes a task, synthesize and play a spoken summary of what happened. Shipped v0.2.0.

F2. Permission-blocked notification — When the agent waits for user approval, announce it. Uses the `Notification` hook with `permission_prompt` subtype. Shipped v0.2.0.

F3. `/vox` command — Enable or disable vox: `y` (chime notifications), `c` (continuous spoken summaries), `n` (off). Shipped v1.0.0.

F4. `/unmute` and `/mute` commands — Toggle voice vs. chime. `/unmute` enables spoken notifications with optional voice selection. `/mute` switches to chime-only. Shipped v0.11.0.

F5. Chime audio — Mood-aware audio tones for chime mode (`/mute`). Eight distinct signals pitch-shift to match session vibe (bright/neutral/dark). 24 total assets. Shipped v0.11.0.

F6. Provider auto-detection — Detect best available TTS provider on first use. ElevenLabs > OpenAI > Polly > macOS `say` > Linux `espeak-ng`. Shipped v0.7.0.

F7. `/recap` command — On-demand spoken summary of the last Claude response. Agent extracts key points and speaks them. Shipped v0.2.0.

F8. Plugin marketplace — Listed in the Claude Code plugin marketplace for one-click install. Shipped v1.2.0.

F9. `voxd` audio daemon — Dedicated audio server with WebSocket protocol. Separates synthesis/playback from MCP server for sub-10ms hook dispatch. Playback queue, dedup, caching. Shipped v3.0.0.

F10. Background music — Vibe-driven instrumental music via ElevenLabs Music API. Adapts to session mood. `/music on|off`. Shipped v4.4.0.

F11. Per-call API key passthrough — Scope a single synthesis call to a specific provider API key for billing isolation. Four secure input paths. Shipped v4.3.0.

F12. Config split — Durable preferences (`vox.md`, tracked in git) separated from ephemeral session state (`vox.local.md`, gitignored). Shipped v4.7.5.

F13. Continuous mode hooks — UserPromptSubmit acknowledgment, SubagentStart/Stop announcements, SessionEnd farewell, PreCompact notification. Shipped v1.4.0.

In Progress

Features actively being built.

F14. Remote voxd connectivity — Client-side env vars `VOXD_HOST`, `VOXD_PORT`, `VOXD_TOKEN` override localhost discovery, enabling cross-host audio playback without SSH tunnels. Server-side `VOXD_BIND` controls the bind address. Token auth secures the connection. Targeting v4.7.7.

Won't Do

Features explicitly excluded. Each exclusion is a deliberate positioning choice.

F15. **Voice input** — Listening to the developer and converting speech to commands. Different product, different architecture (ASR, wake word detection, continuous listening). punt-vox speaks; it does not listen.

F16. **Screen reader / accessibility narrator** — Continuous reading of terminal output for visually impaired users. Requires cursor tracking, ARIA-like semantics, and fundamentally different event architecture. This is valuable but is a different product.

F17. **Agent personality voices** — Character voices, conversational tone, entertainment-oriented inflection. punt-vox does adapt to mood — vibe-driven chime pitch and ElevenLabs expressive tags — but for *information clarity*, not personality. The voice sounds tired after failures because that is useful signal, not because it is playing a character.

F18. **Custom voice training** — Training a TTS model on the user's voice or a custom voice. ElevenLabs supports this, but exposing it adds configuration complexity that contradicts the zero-config design principle.

References

[1] Anthropic Engineering. *Effective Harnesses for Long-Running Agents*. Claude Opus 4: 7+ hours continuous autonomous operation. Security audits 30–60 min; performance profiling 45–90 min. 2026. URL: https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents.

[2] Anthropic. *Measuring AI Agent Autonomy in Practice*. Primary data: median Claude Code turn 45 seconds; 99.9th percentile grew from <25 min to >45 min (Oct 2025–Jan 2026). Auto-approve rate grows from 20% for new users to 40%+ for experienced users (750+ sessions). Claude stops to ask clarification 2x more often than users interrupt on complex tasks. 2026. URL: https://www.anthropic.com/research/measuring-agent-autonomy.

[3] Gloria Mark, Daniela Gudith, and Ulrich Klocke. "The Cost of Interrupted Work: More Speed and Stress". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. Peer-reviewed. 36 knowledge workers. 23 min 15 sec average recovery time after interruption. 2 intervening tasks before returning to original work. 2008. URL: https://ics.uci.edu/~gmark/chi08-mark.pdf.

[4] Stack Overflow. *2025 Stack Overflow Developer Survey — AI Section*. 84% of developers use or plan to use AI tools. 51% use daily. 31% use AI agents. Claude Code used by 41% of agent users. 38% have no plans to adopt agents. 2025. URL: https://survey.stackoverflow.co/2025/ai/.

[5] cfngc4594. *agent-notify: Multi-channel Notifications for AI Coding Agents*. Open-source competitor. Sound, macOS alerts, macOS say TTS, ntfy push. Supports Claude Code, Cursor, Codex. OS TTS only; no premium providers. 2025. URL: https://github.com/cfngc4594/agent-notify.

[6] paulpreibisch. *AgentVibes: Bring Your Claude Code Sessions to Life with Voice*. Open-source competitor. macOS say, Piper TTS, Soprano, Windows SAPI. 914 voices. Background music, personality styles, verbosity levels. No ElevenLabs or OpenAI TTS. 2025. URL: https://github.com/paulpreibisch/AgentVibes.

[7] CNBC. *Nvidia-Backed AI Voice Startup ElevenLabs Hits $11 Billion Valuation*. $500M Series D at $11B valuation. $330M+ ARR end of 2025. 41% Fortune 500 adoption. 2026. URL: https://www.cnbc.com/2026/02/04/nvidia-backed-ai-startup-elevenlabs-11-billion-valuation.html.

[8] TextToLab. *TTS Pricing Comparison 2025*. OpenAI TTS: $15/1M chars (tts-1), $30/1M chars (tts-1-hd). AWS Polly: $4.80/1M standard, $19.20/1M neural. ElevenLabs: subscription from $5/month / 30K chars. 2025. URL: https://texttolab.com/pricing.

[9] TechCrunch. *GitHub Copilot Crosses 20 Million All-Time Users*. 20M cumulative users July 2025, 4x year-over-year. 90% of Fortune 100 use it. 2025. URL: https://techcrunch.com/2025/07/30/github-copilot-crosses-20-million-all-time-users/.

[10] Opsera. *Cursor AI Adoption Trends: Real Data from the Fastest Growing Coding Tool*. Cursor: 1M+ users, 360K paying customers, $500M ARR (May 2025), 50%+ Fortune 500 adoption. 2025. URL: https://opsera.ai/blog/cursor-ai-adoption-trends-real-data-from-the-fastest-growing-coding-tool/.

[11] VentureBeat. *Claude Code's "Tasks" Update Lets Agents Work Longer and Coordinate Across Sessions*. Background tasks recommended for work >30 seconds. Describes Claude Code shift from copilot to background subagent. 2025. URL: https://venturebeat.com/orchestration/claude-codes-tasks-update-lets-agents-work-longer-and-coordinate-across.

[12] Anthropic. *Hooks Reference — Claude Code Documentation*. Official documentation of hook events: Stop (Claude finishes), Notification (permission_prompt, idle_prompt subtypes). Primary source for hook-based notification architecture. 2026. URL: https://code.claude.com/docs/en/hooks.

[13] D. Jung and A. Butz. "Comparing the Effect of Audio and Visual Notifications on Workspace Awareness". In: *Augmented Human Research* (2016). Peer-reviewed. Audio notifications maintain workspace awareness without requiring visual attention. URL: https://link.springer.com/article/10.1007/s41133-016-0003-x.

[14] Courier.com. *Notification Fatigue Is About to Get 10x Worse*. IDC: 80% of enterprise apps will have AI copilots by end of 2026. Context establishing OS notification fatigue. 2026. URL: https://courier-com.medium.com/notification-fatigue-is-about-to-get-10x-worse-60c151909440.